

GUIA DE INTEGRAÇÃO DAS IAs — DO ZERO AOS 100%

M.I.N.D. — Muñdji Intelligent Neural Developer
De uma máquina sem nada a um MIND ON, a responder com modelos reais

Modelos	8 variantes de Qwen2.5-Coder-7B (HuggingFace) + all-MiniLM para o HIPPOCAMPUS
Serviço	5 servidores vLLM (um por modelo distinto), modo openai_compat
Código do MIND	zero alterações — o sistema é agnóstico ao modelo; tudo se faz no .env
"ON" significa	5 servidores de pé + <i>verificar</i> verde + MIND pronto a aceitar tarefas

A SYNAPSE DB e os dados operacionais ficam **sempre locais**. O que sai são os prompts e o código, que vão para os servidores de inferência — a mesma cedência da GPU alugada, consciente.

Passo 0 — Os modelos (verificados no HuggingFace)

Componente	ID no HuggingFace
CORTEX	saidutta69/Qwen2.5-Coder-7B-Instruct-heretic
CEREBELLUM	huihui-ai/Qwen2.5-Coder-7B-Instruct-abliterated
NEURON 1	GodsDevProject/qwe2.5-coder-Uncensored
NEURON 2	Qwen/Qwen2.5-Coder-7B-Instruct (provisório)
NEURON 3	saidutta69/Qwen2.5-Coder-7B-Instruct-heretic
NEURON 4	vanta-research/wraith-coder-7b
NEURON 5	Qwen/Qwen2.5-Coder-7B-Instruct (provisório)
NEURON 6	huihui-ai/Qwen2.5-Coder-7B-Instruct-abliterated
HIPPOCAMPUS	sentence-transformers/all-MiniLM-L6-v2

- **NEURON 2 e 5:** o ID pedido *Qwen/...PyCDistill* NÃO existe. Fica o base Instruct até haver um ID real.
- **NEURON 4:** o ID real leva org — *vanta-research/wraith-coder-7b*.
- **São 5 modelos distintos** (~15 GB cada, ~75 GB no total): não cabem numa só GPU, daí 5 servidores.

Risco a vigiar: os processos *abliterated/heretic/uncensored* podem degradar a aderência ao formato — e o MIND depende dos marcadores *[NEURON_N:linguagem]* e do esquema JSON. Se os modelos divagarem, os ciclos custam mais iterações. O MIND tem redes e mede os desvios; é o que o piloto revela.

Passo 1 — Os cinco servidores

Servidor	Modelo	Serve
A	saidutta69/...heretic	CORTEX, NEURON 3
B	huihui-ai/...abliterated	CEREBELLUM, NEURON 6
C	GodsDevProject/...Uncensored	NEURON 1
D	vanta-research/wraith-coder-7b	NEURON 4
E	Qwen/...Coder-7B-Instruct	NEURON 2, NEURON 5

Cada servidor precisa de GPU com capacidade ≥ 7.0 e ≥ 16 GB (T4, A10, A100...). **A P100 do Kaggle (6.0) não serve** — escolhe "GPU T4 x2".

Passo 2 — Levantar cada servidor

Em cada sessão de GPU, corre o script uma vez com o modelo e os componentes que serve. Exemplo do Servidor A:

```
MODELO=saidutta69/Qwen2.5-Coder-7B-Instruct-heretic \
COMPONENTES="CORTEX NEURON_3" \
python scripts/provisionar_gpu_gratis.py
```

O script verifica a GPU (recusa a P100), instala o vLLM, arranca com `--dtype half` na T4, abre o túnel cloudflared (URL público, protegido por `--api-key`) e **imprime as linhas exactas do .env**:

```
# Servidor de saidutta69/...heretic
CORTEX_ENDPOINT=https://xxxx.trycloudflare.com
CORTEX_TOKEN=
NEURON_3_ENDPOINT=https://xxxx.trycloudflare.com
NEURON_3_TOKEN=
```

Repete para B–E, mudando as variáveis:

Servidor	MODELO=	COMPONENTES=
B	huihui-ai/...abliterated	"CEREBELLUM NEURON_6"
C	GodsDevProject/...Uncensored	"NEURON_1"
D	vanta-research/wraith-coder-7b	"NEURON_4"
E	Qwen/...Coder-7B-Instruct	"NEURON_2 NEURON_5"

Deixa cada sessão a correr. Se parar, o túnel fecha e o URL muda.

Passo 3 — Ligar ao MIND (.env)

Na tua máquina, em `mind/.env`: põe `MODEL_MODE=openai_compat` e cola os blocos dos 5 servidores. Acrescenta os modelos:

```
CORTEX_MODEL=saidutta69/Qwen2.5-Coder-7B-Instruct-heretic
CEREBELLUM_MODEL=huihui-ai/...abliterated
NEURON_1_MODEL=GodsDevProject/qwe2.5-coder-Uncensored
NEURON_2_MODEL=Qwen/Qwen2.5-Coder-7B-Instruct
NEURON_3_MODEL=saidutta69/...heretic
NEURON_4_MODEL=vanta-research/wraith-coder-7b
NEURON_5_MODEL=Qwen/Qwen2.5-Coder-7B-Instruct
NEURON_6_MODEL=huihui-ai/...abliterated
```

O `.env` nunca é versionado — tokens e URLs ficam só na tua máquina.

Passo 4 — Verificar antes de gastar um ciclo

```
python main.py verificar
```

Testa os 8 componentes com um prompt mínimo. Um URL errado ou um token em falta descobre-se aqui em segundos. Espera-se:

```
cortex      OK    ...  -> 'OK'
cerebellum  OK    ...
neuron_1..6 OK    ...
8 a responder, 0 com falha, 0 sem modelo configurado.
```

Se algum falhar, a mensagem traz a razão real da API. **Reportar com o log antes de mexer** — não ajustar às cegas.

Passo 5 — Pôr o MIND a trabalhar (ON)

O MIND é uma ferramenta de linha de comando: corre uma tarefa até ao fim e termina. **"ON" = 5 servidores de pé + verificar verde + MIND pronto a aceitar tarefas.**

Uma tarefa:

```
python main.py "cria um scanner de portas TCP com timeout por porta"
```

O CORTEX cria e distribui, os NEURONS implementam a sua secção, o CEREBELLUM audita, a sandbox executa, e o ciclo repete até ao limiar de aprovação. O resultado fica em *workspace/output/*.

O piloto (mede vários casos de referência):

```
python main.py piloto --max-tarefas 3      # primeira passagem
python main.py piloto                      # as 10 tarefas
```

Passo 6 — O que fica ON, o que corre on-demand

Peça	Estado
5 servidores de modelo	ON contínuo (nas GPUs). Se caem, o MIND não tem com quem falar
SYNAPSE DB	Ficheiro local; persiste entre execuções
Backup (APScheduler)	Corre em paralelo durante a execução; cópias locais. Confirmar que os dados descem antes de desligar as GPUs
Ciclo do MIND	On-demand : arranca por tarefa e termina. Não é um daemon
HIPPOCAMPUS (ML)	Desligado até ~100 ciclos reais. O histórico acumula na mesma

Um MIND a atender pedidos em **contínuo** (serviço em vez de comando por tarefa) é uma camada de servidor à volta do main.py que **ainda não existe** — a interface é só CLI nesta fase, por âmbito. É o passo natural depois de o piloto validar o comportamento.

Passo 7 — Ordem recomendada de arranque

#	Passo
1	Levanta só o Servidor A e faz <i>verificar</i> com CORTEX+NEURON_3 configurados — prova a cadeia com um modelo antes de acender cinco GPUs
2	Corre uma tarefa simples. Observa se marcadores e JSON saem limpos — é aqui que o risco dos modelos uncensored se revela
3	Se estável, levanta os outros quatro, preenche o .env inteiro, <i>verificar</i> a 8/8
4	<i>piloto --max-tarefas 3</i> , depois o piloto completo
5	<i>python main.py ml-status</i> — a conformidade JSON acumulada diz se estes modelos cumprem o contrato de formato. Só depois de medida se preenchem as fichas de model_selection_criteria.md, nunca antes

O MIND fica ON quando os servidores estão de pé e o *verificar* está verde. Daí em diante, cada *python main.py "<tarefa>"* é um ciclo completo com modelos reais — do zero aos 100%.